“Taxmark” - The one workpaper tool you need in your return workflow by Aarvin George Poozhikala

(96% written by human - me; very fun to read)
I started off by investigating the confident mistakes within the ground truth (client packet JSON), which became the basis for building the web-native tool that I would use myself. The mistakes and ground truth told me what to build

Approach and Core Idea: The story of how the ground truth became the screens

Before any code, I had to establish what the ground truth was trying to tell me. The packet and the brief were the starting point of an AI-assisted investigation into its confident mistakes and domain, and that investigation became the basis for everything after it, which helped me gain an intuitive understanding around this specific domain

- **Learning the domain**. My background in fintech gave me a rough sense of the roles of underwriter (preparer), reviewer, the 4-E-PAAS rule, and an e-file signature. But specifically for the context of the Tax Workpaper app for a US firm, I had to understand the shape of the source data model, what confident mistakes meant, delve into the README, and use the brief’s requirements to guide my research in the domain of tax filing and returns
- **Learning the data helped me position the app in the overall tax-filing pipeline.** The input packet’s shape of the schema, the research around the industry's best practices, and the data that was produced by the tax return engine helped me place my Tax Workpaper app appropriately in the overall pipeline. Which led me to formulate these questions: what does the return say? What needs attention (the errors and blockers)? What changed, and who signed it? That became the starting point of crystallizing the UI screens
- **Discovery, mock-ups, thin-deployed slices, and then rounds of improvement.** Ten mockup screens surfaced, showing how the data should be presented, which was eventually rendered down to 3 screens in the final deployment of the product that carries the same level of signal for the end users to interact with. Each screen had to answer the previously stated questions
- **Core Idea.** Every number is a claim that carries its own receipt. Each click on any number should answer these questions: where did it come from (page and field)? What was done to it (the rule and any change with who, why, and the tax effect)? Who checked it (a sign-off bound to the exact value signed), and what is still open?

The findings shaped the data model and errors and how the design surfaced them

- **Confidence could never be proof.** This is why every number in “Taxmark” workpaper app’s current version carries its own source, how it was built, and how it was checked. The confident mistakes that were found looked right on the surface but were still wrong when verified against other sources present in the input packet. 25 of the 27 mistakes surfaced, read as 95% confidence or more. This proved the need for another verification layer, which eventually became this workpaper app Taxmark. All 27 mistakes are listed in **(Appendix F)**
- **One packet, read three ways (Appendix B).** Firstly, an independent (external to the “taxmark” system) AI-assisted investigation of the input packet produced the ledger of the actual 27 confident mistakes that were identified, which also serves as the answer sheet in order to grade the performance of another AI reader within the app’s system. Within the app’s system (backend logic), it takes the input client packet through; 15 families of deterministic checks flag all 27, and for mistakes that live in unstructured wording (client answers, summaries, etc.), an AI reader suggests issues with quotes that code verifies word for word, matching 13 of the 27 and one I had missed (Appendix D). The packet stays read-only, and everything people (end users) do is an append-only event on top of it, which makes the audit trail the data itself
- **How the design surfaced the errors.** The one I would show anyone is California's estimated payments. The return claims 14,000 while the FTB's record shows 10,500, because the engine counted a payment by its date and missed that it was marked "tax year 2024". The page was read at 0.98 to 0.99 confidence, so nothing looked wrong until a second source disagreed. On the Checks screen this is a single row carrying the Blocker, ✦ AI and Confident mistake pills. From there, one click opens the source page, the fix flows through to the

California refund, which drops from 8,256 to 4,756, and the partner signs off on the new number. All 27 mistakes are listed in Appendix F.

Key decisions and tradeoffs

- **Deterministic rules decide, and AI Reader suggests.** The checks are deterministic, logical backend code processing the client packet, which flags blockers in the workpaper flow. While the AI reader working in tandem within the backend code (brain: claude-sonnet-5-5) never blocks filings, a person can accept or dismiss each suggestion it produces. The **trade-off** is that AI still misses about half, which is reported as 13 of 27. The idea is more signals from different sources help the reviewers make better calls
- **Checks are written against the schema, never against the document.** This was one of the most obvious decisions I had to make, because we weren't making this workpaper tool only for one client, but it was made to be reusable. The same checks had to be able to run on the next client’s packet. The **trade-off** is that the 5 of 15 categories (one, for example Agency tie-out: compares estimated payments on the return with what the IRS transcript and FTB account credit for that year.) of checks still depend on the parts of the schema that carry free text and how it is worded. So a client packet that has a different schema can still slip past this, which is where the AI reader is meant to help. Another system design choice is the pull client packet from encrypted storage in order to protect client PII from directly being in the backend
- **An event log over a read-only packet**. History and undo come for free and nothing is ever lost. The **trade-off** is that every request replays the whole log, with each open tab asking again every five seconds, which is fine for one return but would need snapshots across a firm, and a mistake can only be reversed by a new event, never erased.
- **Honest flow-through**. A fix carries through plain sums, so the refund moves the moment the payments are corrected. The **trade-off** is that a line needing a real tax computation keeps the engine's figure and blocks approval until the engine re-runs, and the tax effects shown are estimates from the packet's own rates.
- **A role switcher instead of sign-in**, so one link shows every view while the server still enforces each role. The trade-off is that identity is self-declared: anyone with the link can switch to the partner and sign off, so real sign-in is the first thing to build before a firm uses it.

How I worked with AI for this project

- **On a Meta level (before, outside, and to build the workpaper app).** I have left clues throughout this brief so far on how I used AI throughout the project, but here it is stated more explicitly.
  - Used AI to synthesize and extract ground truth from the hand out materials (pdf, client packet, and readme), which became the grounding for further research into the domain and became the ledger to evaluate the performance of the backend logic and the AI reader
  - I leveraged my own AI engineering SDLC toolkit (publicly available to all) to develop this app. This includes test-driven development, planning, automated reviews, bug reporting, root cause analysis, and AI-assisted code fixing

- **Within the App (AI reader and the AI Harness):** Now even though we have the deterministic checks (15 categories of checks), our backend also looks at the parts of the client's input packet with free-form text using the **AI reader (brain: claude-sonnet-5-5)** and offers suggestions, which get presented to the end users to take action on, not as blockers but as something that draws their attention to certain words. This runs only once for about 1.5 dollars for every version of the ground truth before being pulled by the backend. Now how do we trust the AI reader’s judgment? We use the custom AI harness, which computes the grounding (40 out of 40), recall (17 out of 27), and precision (28 out of 40), which helped tell me how much I can probably trust my AI system running with the backend

Conclusion : Here we are at the end of the brief. I really enjoyed being challenged by this ambiguous problem statement, which led me to make this version of the taxmark work paper. I also want to thank the team from Miraflores for giving me such a well-thought-out handout (case study brief and fake client package)